Projet Fil Rouge :

L'Observatoire Météo

Objectif global

Créer une application robuste où l'on consomme une API externe, on valide ses données entrantes grâce à **Pydantic**, on les stocke dans **SQLite**, et on réexpose le tout via une API moderne **FastAPI** qui documente automatiquement son travail via **Swagger**.

Jour 1 : Le Réseau, le Client HTTP et la Validation avec Pydantic

Objectif : Récupérer des données propres et structurées.

- **Protocoles HTTP & Swagger**
  - Rappel sur le protocole HTTP (méthodes, codes de statut, en-têtes).
  - Exploration de la documentation **Swagger / OpenAPI** de l'API Open-Meteo.
  - Identification des paramètres de requête (*Query Parameters*).
- **Requêtes robustes et Introduction à Pydantic**
  - Utilisation de `requests` avec gestion des erreurs (`try/except`, `raise_for_status`, timeouts) et sessions.
  - **Approfondissement de Pydantic :** Pourquoi valider les données d'une API externe ? (les dictionnaires Python bruts sont fragiles).
  - Création d'un **Modèle Pydantic** (`WeatherResponseModel`) pour s'assurer que les données reçues de l'API Open-Meteo possèdent bien les bons types (température en float, heure en string, etc.) avant de les manipuler.

Jour 2 : Persistance des Données avec SQLite

Objectif : Enregistrer durablement les données validées.

- **Conception et Création de la Base de Données**
  - Introduction aux bases de données relationnelles avec **SQLite**.
  - Modélisation des tables :
    - `villes` (id, nom, latitude, longitude)
    - `releves` (id, ville_id, temperature, vitesse_vent, date_heure)
  - Écriture des scripts SQL de création via le module natif `sqlite3` de Python.
- **Le Pipeline de Stockage**
  - Écriture d'un script complet qui :

    1. Interroge l'API via `requests`.
    2. Valide la réponse avec **Pydantic** (Jour 1).
    3. Insère proprement ces données validées dans **SQLite** (`INSERT`).
  - Écriture de requêtes de lecture pour extraire des statistiques simples de la base.

Jour 3 : Exposition de notre propre API avec FastAPI et Swagger

Objectif : Transformer notre script en un service web documenté.

- **Introduction à FastAPI et réutilisation de Pydantic**
  - Pourquoi FastAPI ? (Asynchrone, ultra-rapide, validation native via Pydantic).
  - Création d'une application FastAPI de base.
  - Réutilisation des **modèles Pydantic** créés au Jour 1 pour structurer les données que notre API va renvoyer au client.
- **Connexion SQLite, Endpoints et Swagger UI**
  - Écriture des routes FastAPI pour interroger notre base de données SQLite (ex: `GET /meteo/{ville}`).
  - **La magie de Swagger :** L'exploration automatique de l'interface interactive générée par FastAPI sur `/docs`.
  - Test des routes, validation des codes de statut HTTP renvoyés par notre propre serveur.
  - *Livrable final :* Une architecture propre, fonctionnelle, où l'on maîtrise tout le cycle de la donnée (du serveur distant jusqu'à notre propre API documentée).